2020 11th International Conference on Information and Communication Systems (ICICS) 978-1-7281-6227-0/20/$31.00 ©2020 IEEE 10.1109/ICICS49469.2020.239548

A One-Cycle Asynchronous FIFO Queue Buffer Circuit

Saleh Abdel-Hafeez[1] and Muhannad Q. Quwaider[2]
[1] Jordan University of Science and Technology, Irbid 22110, Jordan; sabbatical at Qassim University,
Dept. of Computer Eng., College of Computer, Qassim 51542, Saudi Arabia
[2] Jordan University of Science and Technology, Irbid 22110, Jordan
E-mail: [1]s.abdelhafeez@qu.edu.jo, [2] mqquwaider@just.edu.jo

Abstract— The interfaces of process elements in System-On-Chip (SOC) have become a challenge due to process PVT variations that directly harm the clock distribution from one side; and on the other side, the goal to maintain low cost of clock distribution by obviating the use of clock related circuits; such as, PLL and DLL circuits that are generous for consuming power. The first-in first-out (FIFO) memory with asynchronous separate write/read operation is presented to ease the communication between process elements. The aim is to operate the FIFO without signaling like a clock signal (clockless) in a handshake system signaling protocol. This work proposes an energy-efficient asynchronous (clockless) FIFO memory design that operates on the handshake signaling that is *Request* and *Acknowledge* signals. The design generates the *Acknowledge* signal based on the received *Request* signal using a new asynchronous circuit that controls the operation of the FIFO. Thus, the design can be considered as a potential technology of choice for low-power applications such as IoT communication solutions. Compared to prior work, our design can operate 5X faster at the same supply voltage. The design's total power consumption is 2 mW with a total transistor count of 34,470 at 65 nm and 1 V power supply.

Keywords- Asynchronous Circuit; Clockless; FIFO; Handshake; IoT; PVT Variations; SOCs; SRAM 8T-Cell

I. INTRODUCTION

Traditional System-On-Chips (SOCs) have an increasing number of processing elements (PEs) on a single die in order to afford continued performance improvements. The PEs are in a state of communication with each other's, and they are sharing information, forming a kind of network of information among themselves which usually called Network-on-Chip (NoC) [1]-[3]. Conventionally, each PE has a source of generating clock that ranges from a high-cost PLL to low-cost ring oscillator [4]-[5].

The PE's clock source is used to capture the data that is transferred from another PE; Besides, it synchronizes the sending data to another PE. Thus, for non-balanced clock distribution, which is most likely the case due to layout wiring and process-voltage-temperature (PVT) variations [6]-[7]; data might violate the setup/hold time concerning the edge of the clock. Consequently, the communication link is broken between the PE's due to the loss of information.

Moreover, having a high-cost of PLL with prices jitter for each PE as in [8]-[9] is not recommended, especially for a large array of PEs like NoCs since it draws large consumptions power. Other approaches use low-cost clock generator sources like ring-oscillator structure with global high-cost clock distribution sources for a group of PEs [10]-[11]. Others use the topology of globally asynchronous with locally synchronous (GALS) [12]-[14] and used the local source generator to capture the data.

One of the drawbacks of the aforementioned methodologies is the use of multiple clock domains that require high-cost design; besides, multiple clock generators which are commonly known to harvest large consumptions power. Additionally, overall individual clock domains might span many square millimeters, and so many of the design challenges posed by a fully synchronous design persist in these systems.

To address these drawbacks, we present a new architecture for PEs; such that, each PE has asynchronous FIFO. The FIFO has no clock signal, and it operates on receiving the *Request* signal from communication PE. In response, it generates the *Acknowledge* signal to synchronize the write/read operation within the FIFO and to acknowledge the communication PE of receiving the *Request* signal. The proposed FIFO has the asynchronous circuit that orchestrates the read and write operations with communication PE.

We extend prior work on synchronous FIFO designs [15] to include asynchronous read and write operations. The FIFO presented in [15] has several features that suite asynchronous behavior. One of these features is the separation between the memory array and address counter; such that, the memory array can be active for read or write operation while the address counter is holding the address. On the other hand, the address counter is incrementing the address of the memory array, while the memory array is being deactivated, in order to preserve the data while the counter is changing.

Thus, the FIFO operates on mutually exclusive events, which is one of the basic requirements for asynchronous control operations. Other advantages are related to the components' design of the FIFO; such as the Empty-Full flag circuit, which has only four DFFs in regards to the depth of the FIFO. Thus, the design requires minimal changes for scalable memory sizes. Additionally, the memory array is constructed from 8T-Cell that is commonly known for low-power design and high-speed operation with separate ports between read and write. Reference [15] provides further petty details on the FIFO circuit structure.

Authorized licensed use limited to: University of Edinburgh. Downloaded on June 14,2020 at 14:38:54 UTC from IEEE Xplore. Restrictions apply.

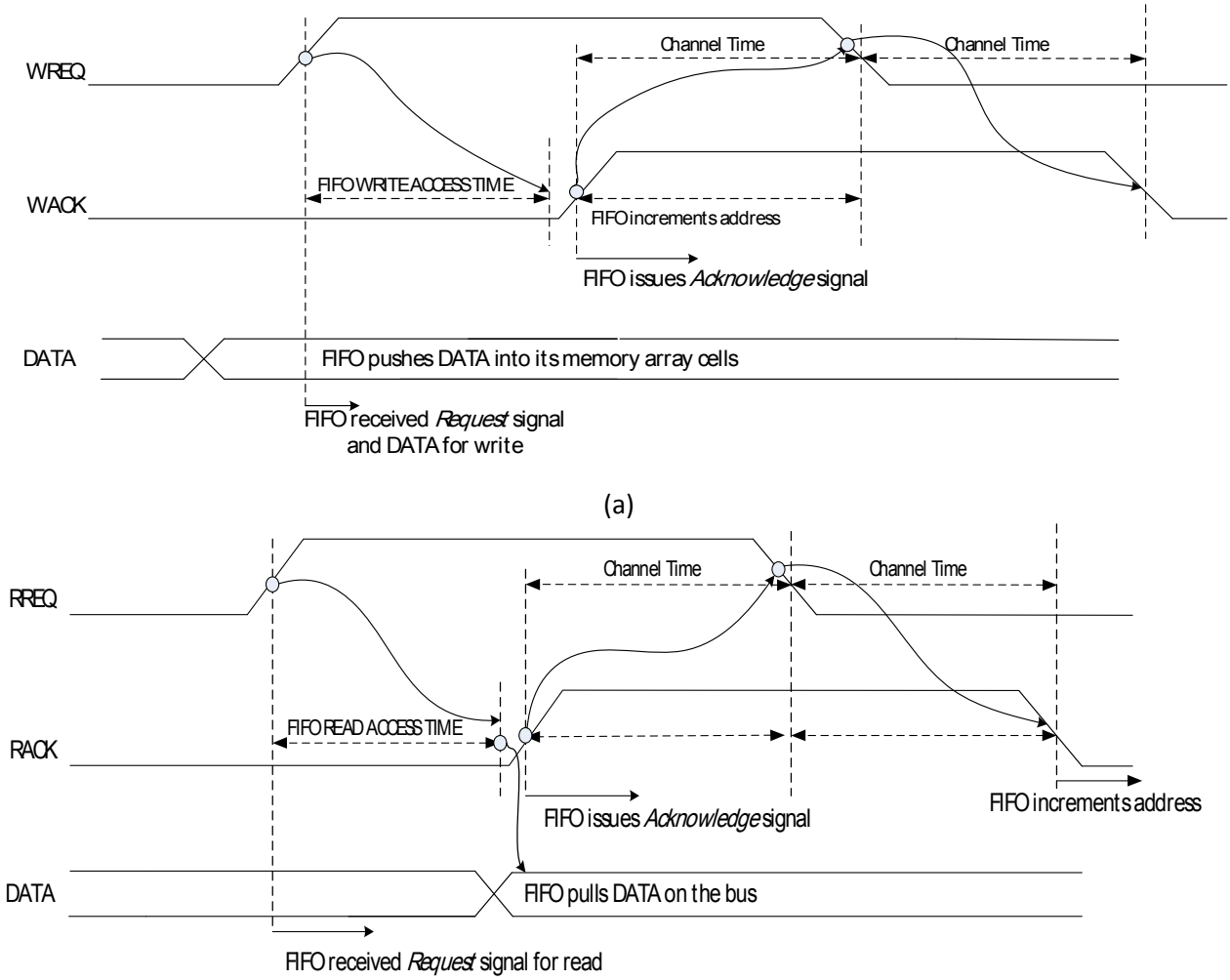

Fig. 1. Asynchronous write operation; (b) Asynchronous read operation

In this proposed work, section II recalls some theories of asynchronous signaling, which are mainly *Request* and *Acknowledge* signals. These two signals are utilized by our FIFO design to trigger the mutually exclusive events, which are related to the circuit components of address counter and memory array. Subsequently, we show how the edges of *Request* and *Acknowledge* signals streamline the events of the FIFO and control the communication with active PE sender. Section III realizes the structure of the circuit for developing asynchronous signals in non-overlapping behavior. Section IV gives the overall proposed asynchronous FIFO hardware design. Additionally, section V illustrates some performance features along with some comparison of recent works. The conclusion is given in Section VI.

II. FIFO WITH ASYNCHRONOUS SIGNALING

Data communication in the asynchronous link is based on a handshake protocol between the sender (active) and receiver (passive) [17]-[18]. The active and passive processing elements are synchronized by handshake *Request* and *Acknowledge* signals. The active element initiates a handshake by issuing a *Request* signal, while the passive element responds by sending an *Acknowledge* signal.

Whenever the sender has data to send, it sends a *Request* signal to the receiver, where the data is encoded along with the edge of the write *Request* signal (WREQ), which is a Push channel (write operation). Consequently, the receiver receives this signal and sends out the write *Acknowledge* signal (WACK) indicating that the data has been received and written. On the other hand, the data can be encoded along with the *Acknowledge* signal, which is a Pull channel (read operation); such that, a sender requests data by sending a read *Request* signal (RREQ). Upon the receiver receives this signal, it sends out the read *Acknowledge* signal (RACK) along with data indicating that the data has been issued.

Consequently, Figure 1(a) depicts the write timing using the Push handshake signaling, while Figure 1(b) depicts the read timing using the Pull handshake signaling. The write scheme (Fig. 1(a)) shows the data (DATA) is accessed from the moment WREQ signal goes high until the moment that the WACK signal goes high; given enough time margin to catch the data by the receiver and write it in FIFO memory array. On the other hand, reading scheme (Fig. 1(b)) shows the read data

Authorized licensed use limited to: University of Edinburgh. Downloaded on June 14,2020 at 14:38:54 UTC from IEEE Xplore. Restrictions apply.

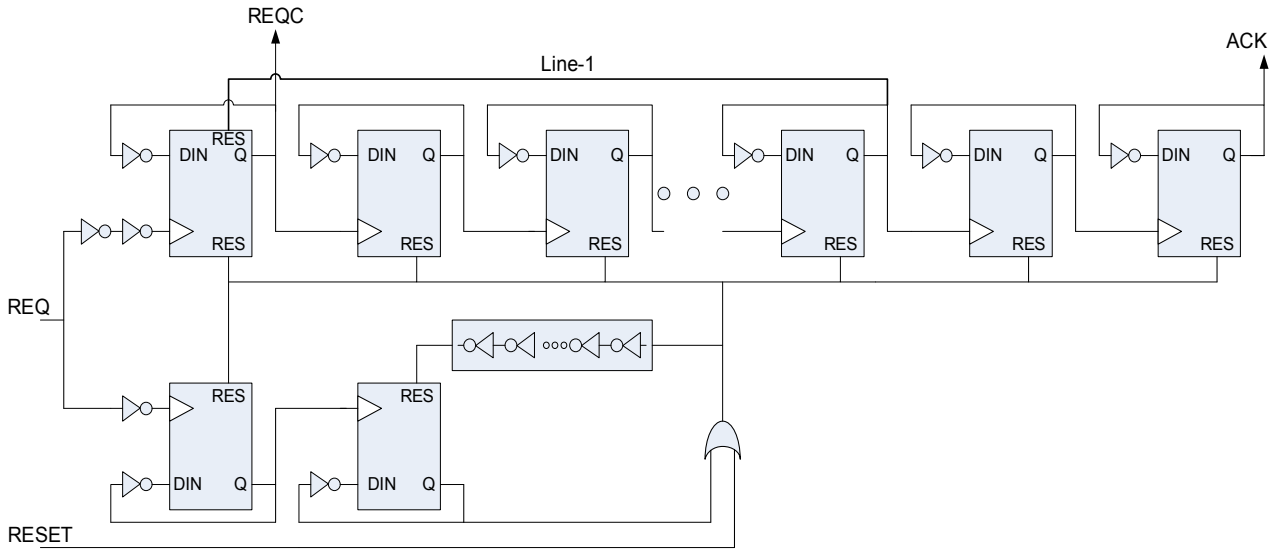

Fig. 2. Asynchronous signaling circuit

is being fetched from the moment the RACK signal goes high until the moment the RACK signal goes high, given enough time margin to fetch the data from the receiver.

Therefore, the rising edge of the WREQ signal triggers the memory array cell to store the data, while the rising edge of WACK closes the memory array and changes the state of the write address counter. Similarly, the timing diagram derived in Fig. 1(b) shows that the rising edge of the RREQ signal triggers the memory array cell to fetch the data on the bus until the rising edge of RACK which ends the memory active array cells. Once the sender receives the RACK signal, it disables the RREQ signal indicating the data has been received by the sender. Then, the receiver disables the RACK signal, and uses this falling edge to change the state of the reading address counter.

III. CIRCUIT DESIGN OF ASYNCHRONOUS SIGNALING

Recall from section II, the *Acknowledge* signal is generated from a passive PE in response to the *Request* signal that is generated from an active PE. Fig. 1 (a)-(b) illustrates similar the timing behavior of *Request* and *Acknowledge* signals for both, read and write, operations. The differences are the actions taking at the edges of *Request* and *Acknowledge* signals.

Therefore, we proposed only one circuit that generates the *Acknowledge* signal with respect to the *Request* signal, and duplicate the circuit for read and write operations. Figure 2 realizes the circuit with conventional D-Type Flip-Flop (DFF) as a delay element that generates the *Acknowledge* signal. The DFF is commonly known for robustness against PVT variations and continued technology scaling [19]-[21] since it relies on the edges of the signal.

Initially, all DFFs are at reset state; that is, all outputs (Qs) are equal "0" value. Once the rising edge of the *Request* (REQ) signal arrives, the clean-request (REQC) signal occurs and is used to activate the FIFO's memory array cells for read or write operation. After the delay chain of DFFs, which is equal to the write or read access time, the REQC signal is reset back to "0" value (i.e. feedback line-1) in order to close the FIFO's memory array cells; such that, no data will be written or read from FIFO. Then, the *Acknowledge* (ACK) signal is activated, and it is propagated across the channel to inform the active PE that the operation has been conducted.

Once the ACK signal reaches the active PE, the PE deactivates the REQ signal and sends it back through the channel to the FIFO (passive element). Once the proposed circuit in Fig. 2 receives the falling edge of the REQ signal, it resets all DFFs, and thus, the ACK signal is deactivated.

During the write operation, the rising edge of ACK is used to increment the address counter of FIFO write decoder since the data has arrived with the REQ signal. Thus, the REQC signal is deactivated first (feedback line-1), then the ACK signal is activated to make sure that these two signals are mutually exclusive events in the FIFO circuit.

However, during the read operation, the falling edge of ACK is used to increment the address counter of FIFO read decoder in order to make sure that the data has arrived at active PE. In contrary, the rising edge of ACK is used to latch the data on the memory output buffers; and thus, on the data bus.

IV. ASYNCHRONOUS FIFO HARDWARE

Figure 3 depicts the architectural hardware block structure for our asynchronous FIFO design as a continuation of our previous coauthored published design [15]. The proposed design exploits two units, which are: the *Datapath* unit and the *Control* unit. TABLE 1 depicts the input/output signals' abbreviations and descriptions.

The *Datapath* unit is constructed of a memory array with two separate read and write ports to provides independent operations using address-counters and address-decoders. Consequently, to avoid memory race and unexpected data operation, the address-counter and address-decoder circuits leverage mutually exclusive timing events; such that, there are no overlapping activities between these two circuits. Thus, our

Authorized licensed use limited to: University of Edinburgh. Downloaded on June 14,2020 at 14:38:54 UTC from IEEE Xplore. Restrictions apply.

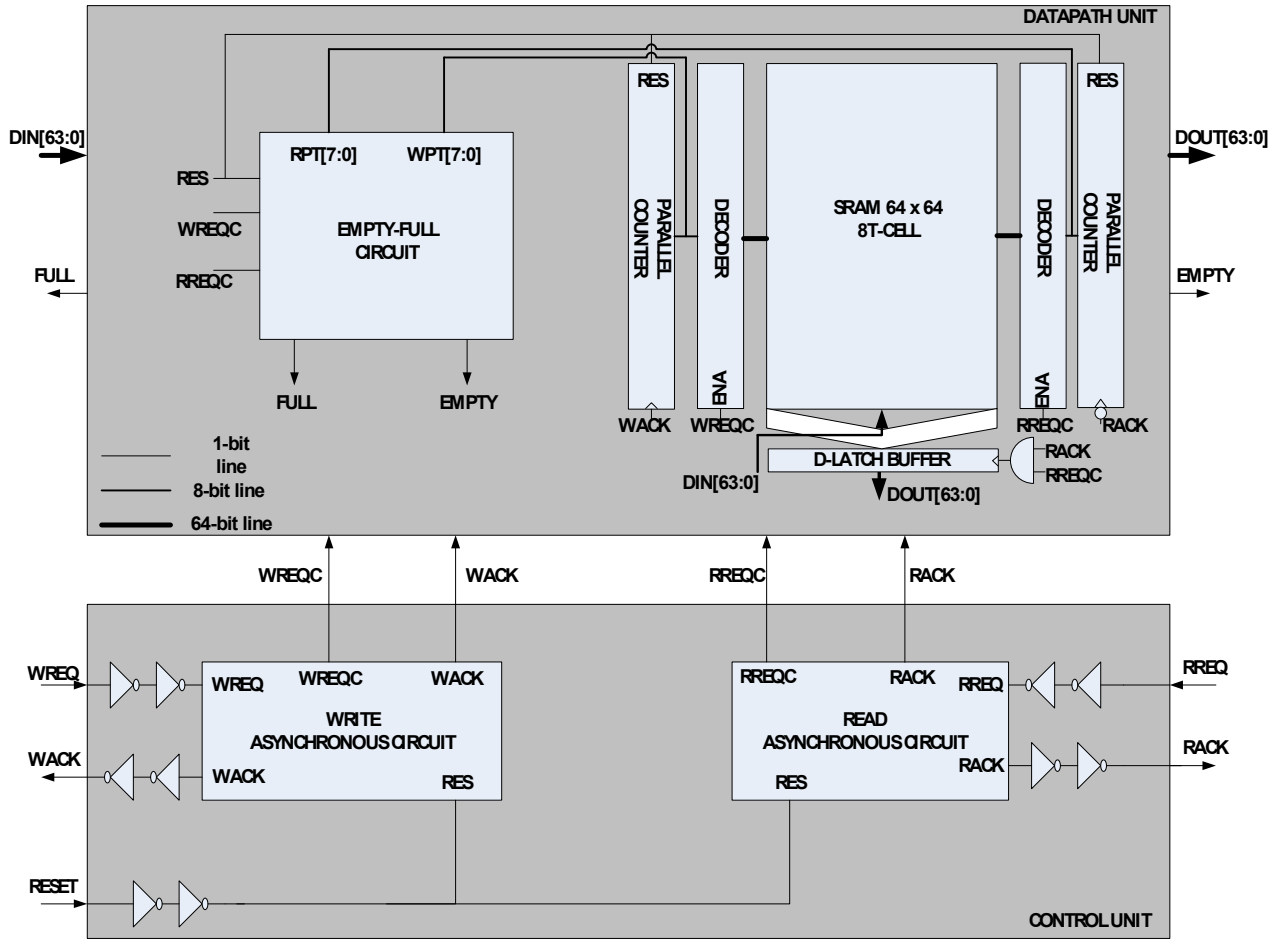

Fig. 1. Detailed hardware diagram for our proposed asynchronous FIFO design

design uses simple and specific distinct signals to activate each circuit.

That is, the write address-counter is triggered by the WREQC signal, while the memory write address-decoder is triggered by WACK signal. Similarly, the read address-counter is triggered by the RREQC signal, while the memory read address-decoder is triggered by the falling edge of the RACK signal. Additionally, the structure's timing of *Request* and *Acknowledge* signals ensures one clock cycle time, without introducing further delay time in a form of latencies.

The control unit constitutes two asynchronous generator circuits that were discussed in Section II. One generator circuit is used for the write operation, while the other generator circuit is used for the read operation as illustrated in Fig. 3. The read asynchronous generator circuit received RREQ signal, and in response, it generates two non-overlapping signals, which are RREQC and RACK to control the read operation of the FIFO and ensures asynchronous handshake operation with active PE.

Similarly, the write asynchronous generator circuit received the WREQ signal, and in response, it generates two non-overlapping signals, which are WREQC and WACK to control the write operation of the FIFO and ensures asynchronous handshake operation with active PE.

Another key advantage is related to *empty-full* flag circuitry which is designed to detect the status of memory in regards to its depth. In other words, the *empty-full* flag circuitry is compensated for different sizes of memory array without adding any extra logic of flip-flop type or re-structuring a new logic circuit. Analogous designs use up-down, ring, or gray code counter that requires further added flip-flops to compensate for any minor changes in memory array size.

TABLE 1
SIGNAL DEFINITIONS

Input-Output Signals	Representation
WREQ	Write enable for the write operation
RREQ	Read enable for the read operation
RACK	Read acknowledge
WACK	Write acknowledge
WREQC	Write clean acknowledge
RREQC	Read clean acknowledge
EMPTY	Flag for the empty buffer
FULL	Flag for the full buffer
DIN [M-1:0]	Input data bus of size *M*
DOUT [M-1:0]	Output data bus of size *M*
RAP [N-1:0]	Read *address-pointer* of size *N*
WAP [N-1:0]	Write *address-pointer* of size *N*

Authorized licensed use limited to: University of Edinburgh. Downloaded on June 14,2020 at 14:38:54 UTC from IEEE Xplore. Restrictions apply.

TABLE 3
COMPARISON BETWEEN PRIOR WORKS AND OUR PROPOSED FIFO DESIGN

	Address-Pointer vs. Memory Decoder	Control Unit Synchronization Type	Storage Unit	Flag Unit	Freq/Voltage /Power/Tech.	Read-Write Timing
[11]	Gray code with control timing circuit	Slow low-cost clock system generator with metastability avoidance circuit	SRAM 8T-Cell 128-word X 16-bit	Up/down indicators	150 MHz/1 V /5.41 mW/28 nm	Three latency delay cycles
[15]	Circular Shifter with one hot decoder	Slow Low-cost clock system generator	SRAM Dual-port 128-word X 32-bit	Johnson counter	28 MHz/ 1V /--/90 nm	Sixteen latency delay cycles
[14]	Cyclic address with one-hot bubble-encoding and shift register	Large cost clock system with GET and PUT control circuits	High-cost Latch-Queue requiring N*(N-1) latches for N lanes	Large cost Domino logic detectors to avoid circuit deadlocks	500 MHz/ 1V -----/ 60 nm	N-1 latency delay cycles
[13]	Token Ring with bubble-encoding	Global island clock system for read and write	High-cost N DFF Buffers	Clock full detector circuit	200 MHz/ 1V/ /----/ 90 nm	N-1 latency delay cycles
Our Work	Carry look-ahead state counter with one-hot decoder	Non-overlapping clockless using 2 opposing phase Handshake signal	8T-Cell SRAM 64-word X 64-bit	Rollover detection and comparison circuit	1 GHz/ 1V / 2mW/ 65 nm	No latency No metastability

V. RESULTS AND ANALYSIS

Without loss of generality and for comparison purposes, we implemented, tested, and verified our asynchronous FIFO hardware architecture of size 64-entry x 64-bits using SRAM 8T-Cells. We architected our proposed asynchronous FIFO at the CMOS transistor level using 65-nm Taiwan Semiconductor Manufacturing Company (TSMC) technology with a 1 V power supply. We gathered timing delay values, total power consumption, and total transistor counts using HSPICE [12] simulations.

The *Datapath* design is activated by two non-overlapping signals, which are RREQC and RACK for the read operation, or WREQC and WACK for the write operation. This non-overlapping reduces the critical path delay by half; besides, it precludes the critical path from accumulating all delays from input-to-output. Thus, the worst-case delay is measured during the half cycle of the above signals.

TABLE 2
SIGNAL DEFINITIONS

Components	CMOS Transistor Counts	Power Consumptions (mW)
Control Unit	72	0.04
Parallel Counter	460	0.27
Address Decoder	384	0.19
Empty-Full Flags	146	0.11
64-word x 64-bit SRAM 8T-Cell	33408	1.3

Simulation results show that our proposed asynchronous FIFO read operations can execute data on the bus at the edge of the read *Request* signal after 20 gate delay time, where each gate has a delay of about 0.02 ns. Consequently, the data can be on the bus after 0.5 ns with respect to the edge of the read *Request* signal. Additionally, the write operation can push data at memory array cells after 12 gate delay with respect to the edge of the write *Request* signal. Thus, the data can be written after 0.25 ns from the edge of the write *Request* signal.

Therefore, the read and write access times are small enough and represent high-speed operations. The proposed control unit provides enough margin for RREQC and WREQC signals that is ranged between 1 ns to 2.5 ns for different process corners. TABLE 2 summarizes the transistors count and power consumption pear each component pertains to our design shown in Fig. 3. The gate delay for the 65-nm technology ranges from 0.015 ns to 0.035 ns depending on the parasitic components and the components' sizes, wherein the minimum technology size for the length is used. For all transistors, we use a channel length of 65 nm where the widths range from 3 µm to 5 µm except the inverter driver widths that are $Wp = 15$ µm and $Wn = 10\ \mu m$. The 8T-Cell SRAM is 65 nm, which is a standard geometry size from Intel [5]. The overall design's transistor count is 34,470 with a total power consumption of approximately 2 mW.

To reduce power consumption, we used SRAM 8T-Cell with 1 V power supply and clockless system, wherein the asynchronous FIFO operates only on the two phases of *Request* and *Acknowledge* signals without any latencies. In contrast, to contemporary designs where the FIFO memory is operated at the internal clock system; such as, a ring oscillator with DLL or PLL; that is widely known to harvest large power consumptions.

Authorized licensed use limited to: University of Edinburgh. Downloaded on June 14,2020 at 14:38:54 UTC from IEEE Xplore. Restrictions apply.

TABLE 3 shows the characteristics and component architecture details for our FIFO compared to related works [11],[13],[14],[15] for continued technology scaling. All compared designs use clock system to synchronize the write and read operations, wherein some designs [11],[15],[14] generate the clock internally with low-cost circuit, while other designs; such as, [13] use the global isolated island for GLAS to synchronize the read and write operations.

The problem of using clock is the alignment between data and the edge of the clock with the existence of PVT, jitter, and layout scaling. Thus, pioneers try to introduce several latencies to avoid metastability and data erroneous behavior. In contrary, our FIFO design has well-defined timing which operates on two phases of non-overlapping signals. Thus, the design provides low power consumption since it relies only on the strobe signals of *Request* and *Acknowledge,* and there is no toggling of the clock. Compared to prior work, our design can operate 5X faster at the same supply voltage. Most important, the consumption power at maximum frequency is only 2 mW about 5X less than counterpart design. The proposed design is latency-free and has no metastability issues.

V. CONCLUSIONS

Our proposed asynchronous FIFO is a clockless design that relies only on the strobe signals of *Request* and *Acknowledge*. Two novel circuits are proposed for read and write operations in order to generate the *Acknowledge* signal based on the received *Request* signal. The FIFO is tailored to operate on these two signals with no clocking system. The design has a well-defined timing structure avoiding metastability and latency delay. The FIFO can access data from/to bus in 0.5ns with 2 mW operation activities. The proposed asynchronous FIFO design can be a potential structure for IoT handshake and NoCs communications.

VI. ACKNOWLEDGEMENTS

The authors would like to acknowledge the support of the Jordan University of Science and Technology for providing the financial support to complete this work.

VII. REFERENCES

[1] A. J. Martin and M. MystrÖm, “Asynchronous Techniques for System-on-Chip Design”, Proceedings of the IEEE, Vol. 94, pp. 1089-1120, 2006.

[2] R. Ezz-Edin, M. A. El-Moursy, H. F. A. Hamed, “High Throughput Asynchronous NoC Design under High Process Variation”, Integration, the VLSI journal, Vol. 49, pp. 1-13, 2015

[3] A. Sheibanyrad, A. Greiner, “Two Efficient Synchronous – Asynchronous converters well-suited for Networks-on-Chip in GALS architectures”, Integration, the VLSI journal, Vol. 41, pp. 17-26, 2008

[4] S. Furber. ARM: system-on-chip architecture, 2nd edition, Addison-Wesley, 2000.

[5] S. Abdel-hafeez, S. M. Harb and K. M. Lee, “On-Chip Jitter Measurement architecture using a delay-Locked Loop with Vernier Delay Line to the order of Giga Hertz”, Proceedings of the 18th International Conference Mixed Design of Integrated Circuits and System (MIXDES), June 2011.

[6] B. Nowacki, N. Paulino, and J. Goes. A Simple 1 GHz Non-Overlapping Two-Phase Clock Generators for SC Circuits. MIXDES 2013, 20th International Conference "Mixed Design of Integrated Circuits and Systems", June 20-22, Gdynia, Poland, pp. 174-178, 2013

[7] D. Zhang , HG Yang , W. Zhu , W. Li, Z. Huang , L. Li, and T. Li. A Multiphase DLL With a Novel Fast-Locking Fine-Code Time-to-Digital Converter. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, Volume: 23 , Issue: 11, pp. 2680 – 2684, 2015

[8] W. Lin and W. C. Black, Jr. A low-jitter skew-calibrated multi-phase clock generator for time-interleaved applications. Solid-State Circuits Conference, 2001. Digest of Technical Papers. ISSCC. 2001, IEEE International, pp. 396-397, 2001

[9] IEEE spectrum, Breaking the Multicore Bottleneck, Nov. 2016

[10] Y. Bae, S. Park, and I. Park. A single-chip programmable platform based on a Multithreaded Processor and Configurable logic Clusters, IEEE Journal of Solid-State Circuits, vol. 38, No. 10, Oct. 2003.

[11] B. Keller, M. Fojtik, and B. Khailany. A Pausible Bisynchronous FIFO for GLAS systems. 21st IEEE International Symposium on Asynchronous Circuits and Systems, pp. 1-8, California, May 2015

[12] A. Rahmani, P. Liljeberg, J. Plosila, and H. Tenhunen. Design and implementation of reconfigurable FIFOs for Voltage/Frequency Island-based Networks-on-Chip. Microprocessors and Microsystems, Vol. 37, pp. 432-445, June–July 2013.

[13] I. M. Panades and A. Greiner, “Bi-Synchronous FIFO for Synchronous Circuit Well Suited for Network-on-Chip in GALS architectures”, Proceedings of the First International Symposium on Networks-on-Chip (NOCS’07), May 2007.

[14] T. Chelcea and S. M. Nowick, “A Low-Latency FIFO for Mixed-Clock Systems”, Proceedings IEEE Computer Society Workshop on VLSI 2000. System Design for a System-on-Chip Era, April 2000

[15] M. Fattah, A. Manian, A. Rahimi and S. Mohammadi, “A High Throughput Low Power FIFO used for GALS NoC Buffers”, IEEE Computer Society Annual Symposium on VLSI, July 2010

[16] A. Gordon-Ross, S. Abdel-hafeez, M. H. Alsafrjalni, “A one-Cycle FIFO Buffer for Memory Management Units in Manycore System”, IEEE Computer Society Annual Symposium on VLSI, July 2019

[17] A. Peeters and K. V. Berkel, “Signle-Rail Handshake Circuits”, IEEE Journal of Solid-State Circuits, pp. 1429-1439, 1999

[18] Joep Kessels, “Register-Communication Between Mutually Asynchronous Domains”, IEEE International Symposium on Asynchronous Circuits and Systems, March 2005

[19] S. Abdel-Hafeez and A. Gordon-Ross. A Digital CMOS Parallel Counter Architecture Based on State Look-Ahead logic, Journal of IEEE Transactions on Very Large Scale Integration (VLSI) Systems, Vol. 19, Issue 6, , pp. 1023-1034, May 23, 2011.

[20] S. Abdel-Hafeez, M. Shatnawi, and A. Gordon-Ross, A DOUBLE DATA RATE 8T-CELL SRAM ARCHITECTURE FOR SYSTEMS-ON-CHIP", IEEE 14Th International Symposium on System-on-Chip 2012, Tampere, Finland, October 11-12, 2012.

[21] S. Abdel-hafeez, S. Otoom, and M. Quwaider, “Alias Table Memory Circuit For Register Renaming Unit,” in 2019 10th International Conference on Information and Communication Systems (ICICS), 2019, pp. 262–267.

Authorized licensed use limited to: University of Edinburgh. Downloaded on June 14,2020 at 14:38:54 UTC from IEEE Xplore. Restrictions apply.